

California

Julio 21, 2026



Christian Villegas

Link del repositorio: <https://github.com/christian7villegas/Introducci-nPython>

```
In [4]: #Iportamos las librerias de a utilizar
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, Lasso, LinearRegression
from sklearn.preprocessing import StandardScaler
import zipfile
```

```
In [14]: import os

os.environ["KAGGLE_CONFIG_DIR"] = r"C:\Users\ESTUDIANTES-ITQ\.kaggle"
```

```
In [15]: #Creamos la carpeta dataset
os.makedirs("dataset1", exist_ok=True)
```

```
In [16]: #Descargamos el dataset
!kaggle datasets download -d camnugent/california-housing-prices
```

Dataset URL: <https://www.kaggle.com/datasets/camnugent/california-housing-prices>
License(s): CC0-1.0
Downloading california-housing-prices.zip to C:\Users\ESTUDIANTES-ITQ\Desktop\machine learning\practicass

```
0%|          | 0.00/400k [00:00<?, ?B/s]
100%|#####| 400k/400k [00:00<00:00, 1.32MB/s]
100%|#####| 400k/400k [00:00<00:00, 1.32MB/s]
```

```
In [18]: #Descomprimos el archivo
with zipfile.ZipFile("california-housing-prices.zip", 'r') as zip_ref:
    zip_ref.extractall("dataset1")
```

```
In [19]: #Cargamos el conjunto de datos
df = pd.read_csv("dataset1/housing.csv")
#Leemos las primera filas
print ("Primera filas")
display(df.head())
```

Primera filas

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0

```
In [20]: #Información general del dataset
print("\nInformación general de dataset:")
df.info()
```

Información general de dataset:

<class 'pandas.DataFrame'>

RangeIndex: 20640 entries, 0 to 20639

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	longitude	20640 non-null	float64
1	latitude	20640 non-null	float64
2	housing_median_age	20640 non-null	float64
3	total_rooms	20640 non-null	float64
4	total_bedrooms	20433 non-null	float64
5	population	20640 non-null	float64
6	households	20640 non-null	float64
7	median_income	20640 non-null	float64
8	median_house_value	20640 non-null	float64
9	ocean_proximity	20640 non-null	str

dtypes: float64(9), str(1)

memory usage: 1.6 MB

```
In [21]: #Exploramos las estadísticas descriptiva
print("\nEstadísticas descriptivas del dataset:")
display(df.describe())
```

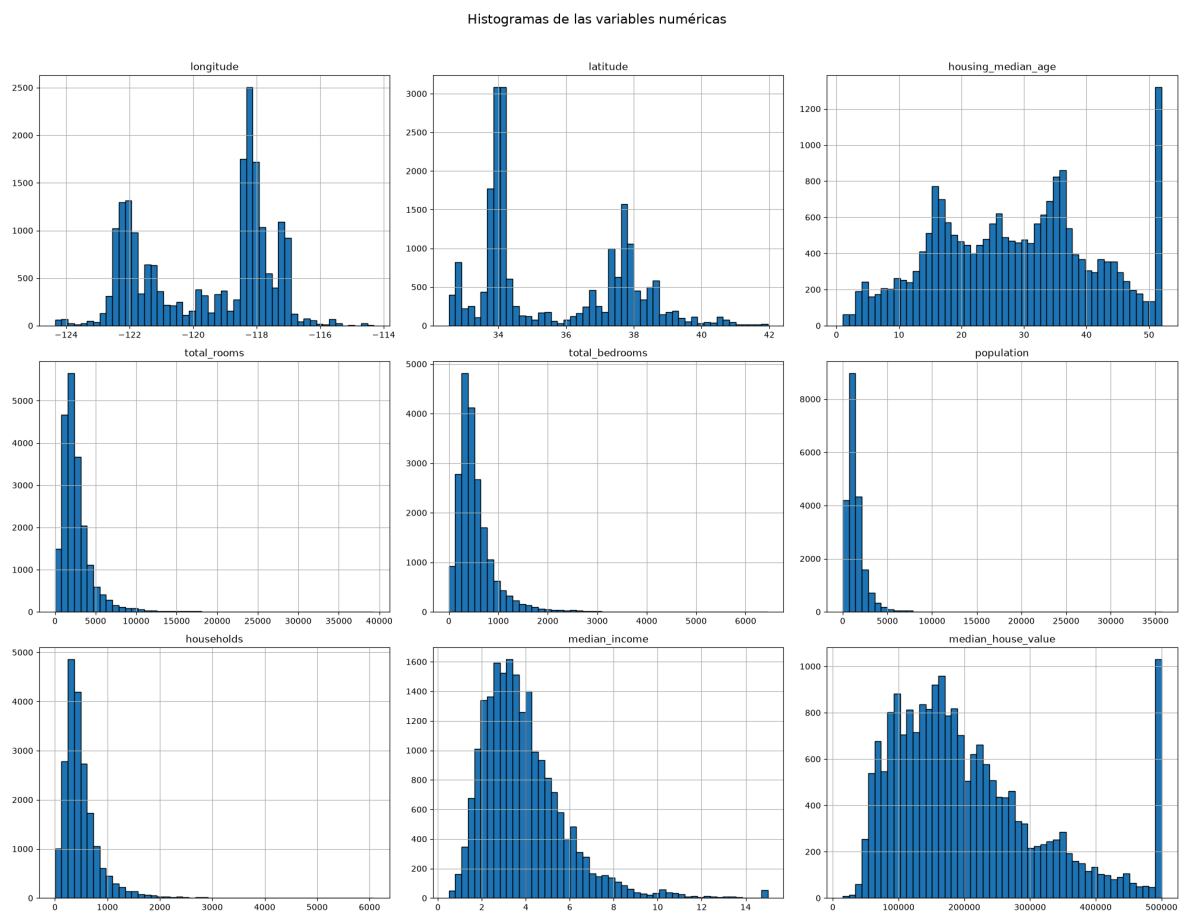
Estadísticas descriptivas del dataset:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	popul
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.00
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.47
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.46
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.00
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.00
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.00
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.00
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.00

2. Análisis Exploratorio de datos EDA

Distribución de variable numéricas

```
In [22]: df.hist(figsize=(20,15), bins=50, edgecolor='black')
plt.suptitle("Histogramas de las variables numéricas", y=1.02, fontsize=16)
plt.tight_layout()
plt.show()
```



Limpieza y procesamiento de datos

```
In [23]: # Imputar los valores faltantes en 'total_bedrooms' con la mediana
mediana_dormitorios = df['total_bedrooms'].median()
df['total_bedrooms'] = df['total_bedrooms'].fillna(mediana_dormitorios)

# Eliminación de topes artificiales ('median_house_value' < 500k y 'housing_median_age' < 52)
df_clean = df[(df['median_house_value'] < 500000) & (df['housing_median_age'] < 52.0)].copy()

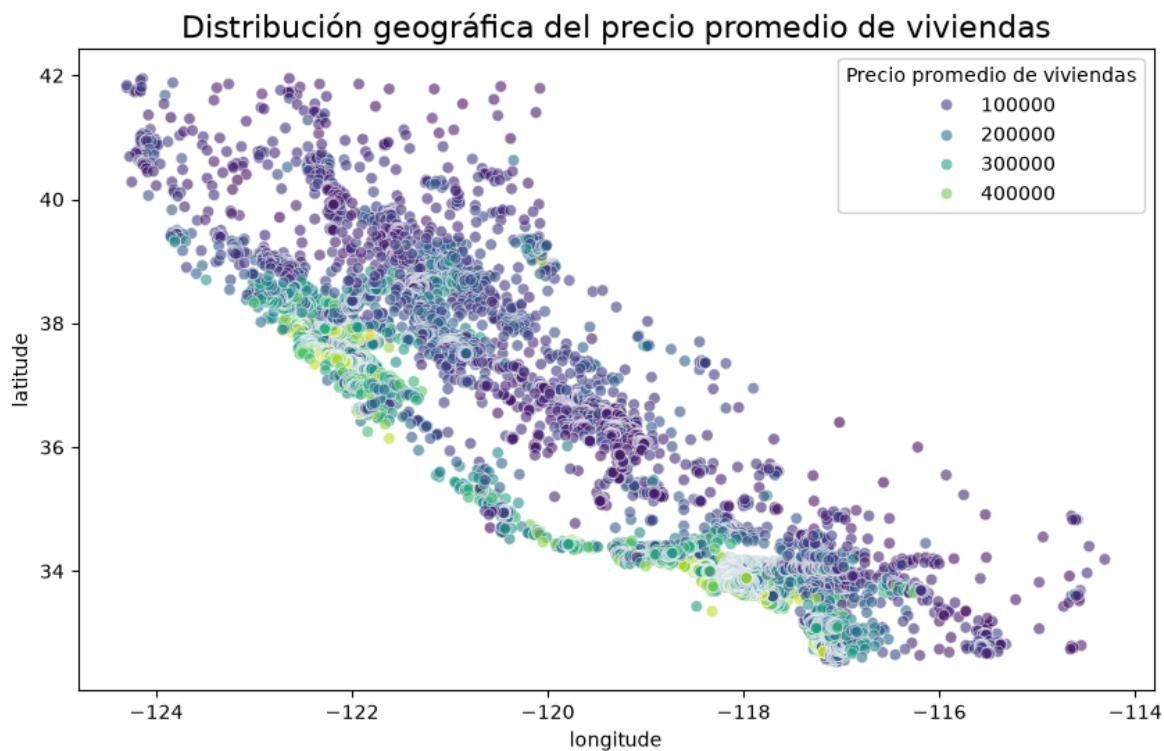
# Eliminar valores físicamente imposibles, mas dormitorios que habitaciones
df_clean = df_clean[df_clean['total_bedrooms'] <= df_clean['total_rooms']]
```

```
In [24]: # Visualizo la información del data set
df_clean.info()
```

```
<class 'pandas.DataFrame'>
Index: 18551 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              18551 non-null  float64
1   latitude               18551 non-null  float64
2   housing_median_age     18551 non-null  float64
3   total_rooms            18551 non-null  float64
4   total_bedrooms         18551 non-null  float64
5   population             18551 non-null  float64
6   households             18551 non-null  float64
7   median_income          18551 non-null  float64
8   median_house_value     18551 non-null  float64
9   ocean_proximity        18551 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.6 MB
```

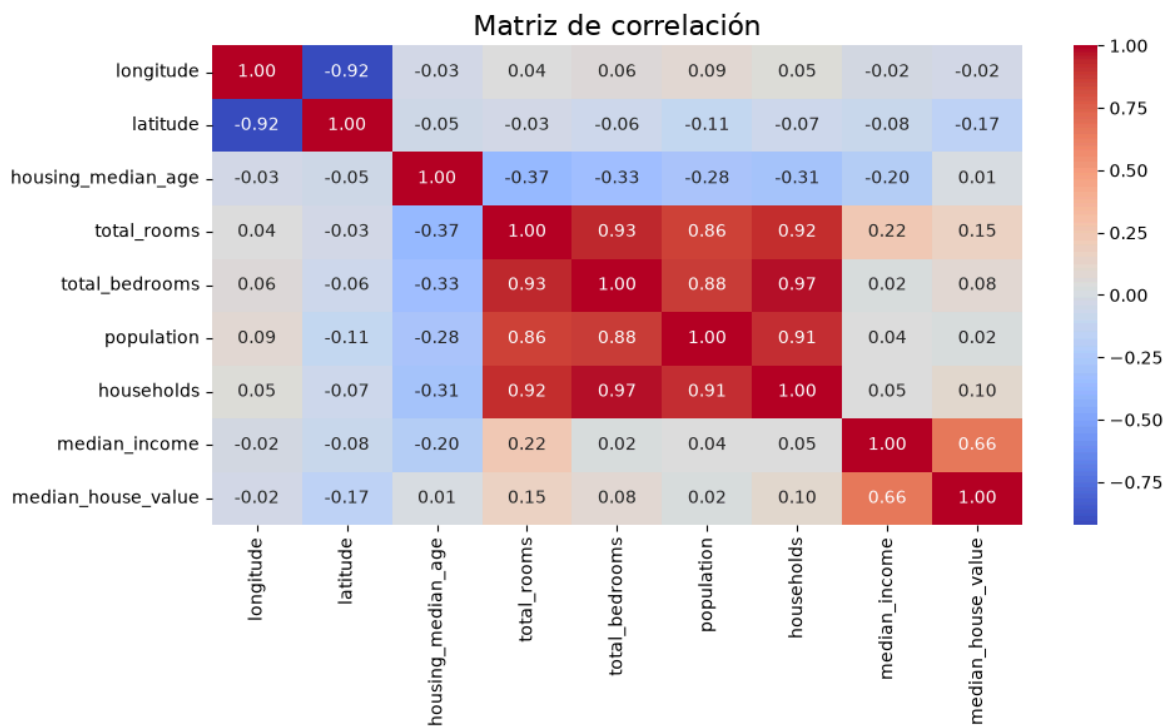
2.2 Visualización Espacial

```
In [26]: plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_clean, x='longitude', y='latitude', hue='median_house_value', palette='viridis', alpha=0.5)
plt.legend(title='Precio promedio de viviendas', loc='upper right')
plt.title('Distribución geográfica del precio promedio de viviendas', fontsize=16)
plt.show()
```



2.3 Análisis de correlación

```
In [27]: plt.figure(figsize=(10, 6))
sns.heatmap(df_clean.corr(numeric_only=True), annot=True, cmap='coolwarm',fmt=".2f")
plt.title('Matriz de correlación', fontsize=16)
plt.tight_layout()
plt.show()
```



Ingenieria de caracterisiticas

```
In [29]: LA, SF = (34.0522, -118.2437), (37.7749, -122.4194)
```

```

# Creación de variables geográficas (Proximidad a centros económicos).funcion de distancia euclínea
df_clean['dist_to_LA'] = np.sqrt((df_clean['latitude'] - LA[0])**2 + (df_clean['longitude'] - LA[1])**2)
df_clean['dist_to_SF'] = np.sqrt((df_clean['latitude'] - SF[0])**2 + (df_clean['longitude'] - SF[1])**2 )

# Ratios
# Vamos a calcular el promedio de dormitorios por habitaciones
df_clean['bedrooms_per_room'] = df_clean['total_bedrooms'] / df_clean['total_rooms']

#Cunatas personas viven por cada casa
df_clean['pop_per_household'] = df_clean['population'] / df_clean['households']

# Encoding de variables categoricas
df_clean = pd.get_dummies(df_clean, columns=['ocean_proximity'],drop_first=True)

```

Analisis de Correlación de las nuevas variables

```

In [30]: corr_matriz= df_clean.corr()
print("\nCorrelación respecto al precio promedio de viviendas:")
display(corr_matriz['median_house_value'].sort_values(ascending=False))

```

Correlación respecto al precio promedio de viviendas:

```

median_house_value      1.000000
median_income            0.664341
total_rooms             0.153769
ocean_proximity_NEAR OCEAN 0.147346
ocean_proximity_NEAR BAY  0.111688
households              0.100436
total_bedrooms          0.079249
population              0.022399
ocean_proximity_ISLAND   0.019534
housing_median_age      0.014207
dist_to_SF              -0.015476
pop_per_household       -0.020200
longitude               -0.022912
dist_to_LA              -0.143450
latitude                -0.171567
bedrooms_per_room       -0.227051
ocean_proximity_INLAND  -0.501263
Name: median_house_value, dtype: float64

```

Preparación de Datos para el Modelado

```

In [31]: # Definir las variables predictoras y la variable objetivo
X = df_clean.drop('median_house_value', axis=1)
y = df_clean['median_house_value']

# Dividir el dataset en conjunto de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Escalar las características
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

In [34]: #print(X)
#print(y)

```

Modelado y Entrenamiento

```
In [35]: # Instanciar y entrenar el modelo de regresión lineal
linear_model=LinearRegression()
ridge_model = Ridge(alpha=10.0, random_state=42) # alpha es la fuerza de regularización
lasso_model = Lasso(alpha=50, random_state=42, max_iter=10000)

#Entrenamos los modelos de aprendizaje
linear_model.fit(X_train_scaled, y_train)
ridge_model.fit(X_train_scaled, y_train)
lasso_model.fit(X_train_scaled, y_train)
```

```
Out[35]:
```

▼ Lasso ⓘ ?

► Parameters

► Fitted attributes

Regresión lineal Standard

```
In [36]: # Predicciones entrenamiento
y_train_pred_linear = linear_model.predict(X_train_scaled)

# Predicciones prueba
y_test_pred_linear = linear_model.predict(X_test_scaled)

# Cálculo de RMSE. Formula: RMSE: sqrt(mean((y_true - y_pred)^2))
rmse_train_linear = np.sqrt(np.mean((y_train - y_train_pred_linear) ** 2))
rmse_test_linear = np.sqrt(np.mean((y_test - y_test_pred_linear) ** 2))

#Calculo de MAE
mae_train_linear = np.mean(np.abs(y_train - y_train_pred_linear))
mae_test_linear = np.mean(np.abs(y_test - y_test_pred_linear))

#Mosttramos los resultados
print(f'Linear - RMSE Train: {rmse_train_linear:.2f}')
print(f'Linear - RMSE Test: {rmse_test_linear:.2f}')

print(f'Linear - MAE Train: {mae_train_linear:.2f}')
print(f'Linear - MAE Test: {mae_test_linear:.2f}')

#Importancia de variables
coef_linear = pd.DataFrame({'Característica': X.columns, 'Importancia': linear_model.coef_.round(3)})
coef_linear = coef_linear.sort_values(by='Importancia', key=abs, ascending=False)

display(coef_linear.head(10))
```

```
Linear - RMSE Train: 57330.01
Linear - RMSE Test: 57261.37
Linear - MAE Train: 41955.67
Linear - MAE Test: 42267.33
```

	Característica	Importancia
7	median_income	61997.456
5	population	-38464.037
6	households	29993.851
0	longitude	-26311.551
8	dist_to_LA	-23266.464
12	ocean_proximity_INLAND	-21393.725
1	latitude	-17693.367
9	dist_to_SF	-15731.574
10	bedrooms_per_room	11404.580
3	total_rooms	7104.186

Modelo Ridge

```
In [38]: # Predicciones
y_train_pred_ridge = ridge_model.predict(X_train_scaled)
y_test_pred_ridge = ridge_model.predict(X_test_scaled)

# Cálculo de RMSE. Formula: RMSE: sqrt(mean((y_true - y_pred)^2))
rmse_train_ridge = np.sqrt(np.mean((y_train - y_train_pred_ridge) ** 2))
rmse_test_ridge = np.sqrt(np.mean((y_test - y_test_pred_ridge) ** 2))

print(f'Ridge - RMSE Train: {rmse_train_ridge:.2f}')
print(f'Ridge - RMSE Test: {rmse_test_ridge:.2f}')

mae_train_ridge=np.mean(np.abs(y_train - y_train_pred_ridge))
mae_test_ridge=np.mean(np.abs(y_test - y_test_pred_ridge))

print(f'MAE Train: {mae_train_ridge:.2f}')
print(f'MAE Test: {mae_test_ridge:.2f}')

# Ver qué variables tienen coeficientes más grandes en el modelo Ridge
coeficientes = pd.DataFrame({ 'Característica': X.columns,'Importancia': ridge_model.coef_.round(3)}).sort_values(
by='Importancia', ascending=False)

print("\nImportancia de las características según el modelo Ridge:")
display(coeficientes.head(10))
```

Ridge - RMSE Train: 57330.29

Ridge - RMSE Test: 57255.13

MAE Train: 41958.59

MAE Test: 42264.60

Importancia de las características según el modelo Ridge:

	Característica	Importancia
7	median_income	61934.472
5	population	-38153.471
6	households	29423.475
0	longitude	-25702.655
8	dist_to_LA	-23197.983
12	ocean_proximity_INLAND	-21555.084
1	latitude	-17313.545
9	dist_to_SF	-15949.976
10	bedrooms_per_room	11325.072
3	total_rooms	6999.325

Modelo LAsso (Regularización L1)

```
In [41]: y_train_pred_lasso = lasso_model.predict(X_train_scaled)
y_test_pred_lasso = lasso_model.predict(X_test_scaled)

rmse_train_lasso = np.sqrt(np.mean((y_train - y_train_pred_lasso) ** 2))
rmse_test_lasso = np.sqrt(np.mean((y_test - y_test_pred_lasso) ** 2))

print(f'Lasso - RMSE Train: {rmse_train_lasso:.2f}')
print(f'Lasso - RMSE Test: {rmse_test_lasso:.2f}')

mae_train_lasso=np.mean(np.abs(y_train - y_train_pred_lasso))
mae_test_lasso=np.mean(np.abs(y_test - y_test_pred_lasso))

print(f'MAE Train: {mae_train_lasso:.2f}')
print(f'MAE Test: {mae_test_lasso:.2f}')

# Ver qué variables tienen coeficientes más grandes en el modelo Lasso
coeficientes_lasso = pd.DataFrame({
    'Característica': X.columns,
    'Importancia': lasso_model.coef_.round(3)
}).sort_values(by='Importancia', key=abs, ascending=False)

print("\nImportancia de las características según el modelo Lasso:")
display(coeficientes_lasso.head())

# Ver si lasso ha eliminado alguna variable (coeficiente exactamente 0)
variables_eliminadas = coeficientes_lasso[coeficientes_lasso['Importancia'] == 0]['Característica'].tolist()
print(f'\nVariables eliminadas por Lasso (coeficiente 0): {variables_eliminadas}')
```

```
Lasso - RMSE Train: 57331.80
Lasso - RMSE Test: 57253.64
MAE Train: 41953.47
MAE Test: 42260.12
```

Importancia de las características según el modelo Lasso:

	Característica	Importancia
7	median_income	61978.503
5	population	-37742.180
6	households	29465.904
0	longitude	-24923.557
8	dist_to_LA	-23182.754

Variables eliminadas por Lasso (coeficiente 0):[]

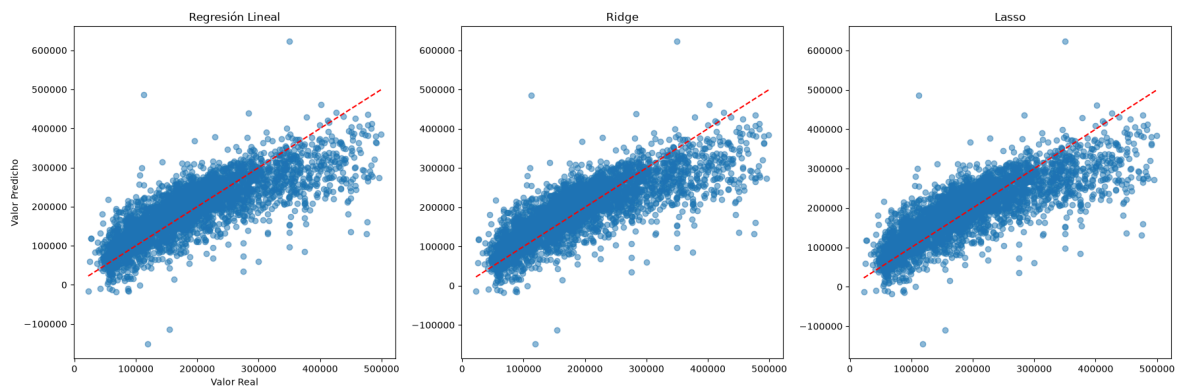
Graficas Valores Reales vs Predichos

```
In [42]: fig, ax = plt.subplots(1, 3, figsize=(18,6))
#Linear
ax[0].scatter(y_test, y_test_pred_linear, alpha=0.5)
ax[0].plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)
ax[0].set_title("Regresión Lineal")
ax[0].set_xlabel("Valor Real")
ax[0].set_ylabel("Valor Predicho")

#Ridge
ax[1].scatter(y_test, y_test_pred_ridge, alpha=0.5)
ax[1].plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)
ax[1].set_title("Ridge")

#Lasso
ax[2].scatter(y_test, y_test_pred_lasso, alpha=0.5)
ax[2].plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)
ax[2].set_title("Lasso")

plt.tight_layout()
plt.show()
```



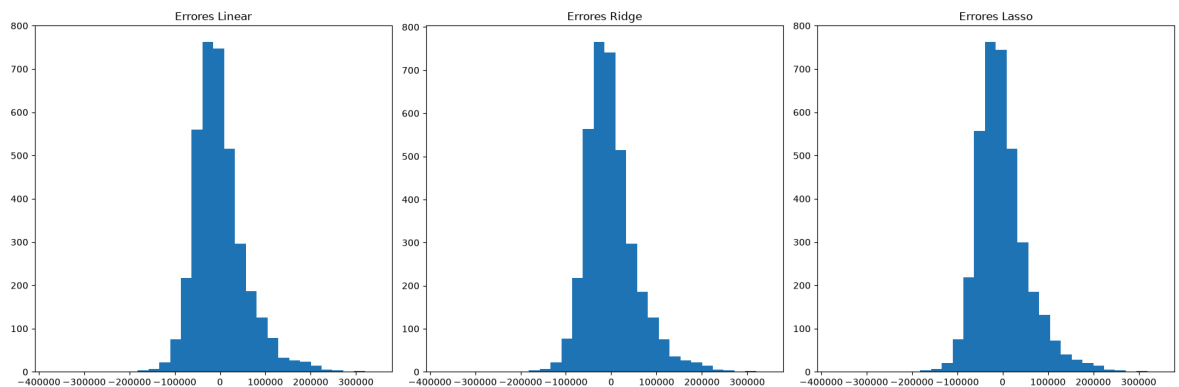
```
In [45]: #Histograma de error
fig, ax = plt.subplots(1,3, figsize=(18,6))

# Linear
ax[0].hist(
    y_test - y_test_pred_linear,
    bins=30
)
ax[0].set_title("Errores Linear")

# Ridge
ax[1].hist(
    y_test - y_test_pred_ride,
    bins=30
)
ax[1].set_title("Errores Ridge")

# Lasso
ax[2].hist(
    y_test - y_test_pred_lasso,
    bins=30
)
ax[2].set_title("Errores Lasso")

plt.tight_layout()
plt.show()
```



```
In [47]: fig, ax = plt.subplots(1,3, figsize=(18,6))

# Linear
res_linear = y_test - y_test_pred_linear

ax[0].scatter(
    y_test_pred_linear,
    res_linear,
```

```

    alpha=0.5
)

ax[0].axhline(0, color='red', linestyle='--')
ax[0].set_title("Residuos Linear")

# Ridge
res_ridge = y_test - y_test_pred_ridge

ax[1].scatter(
    y_test_pred_ridge,
    res_ridge,
    alpha=0.5
)

ax[1].axhline(0, color='red', linestyle='--')
ax[1].set_title("Residuos Ridge")

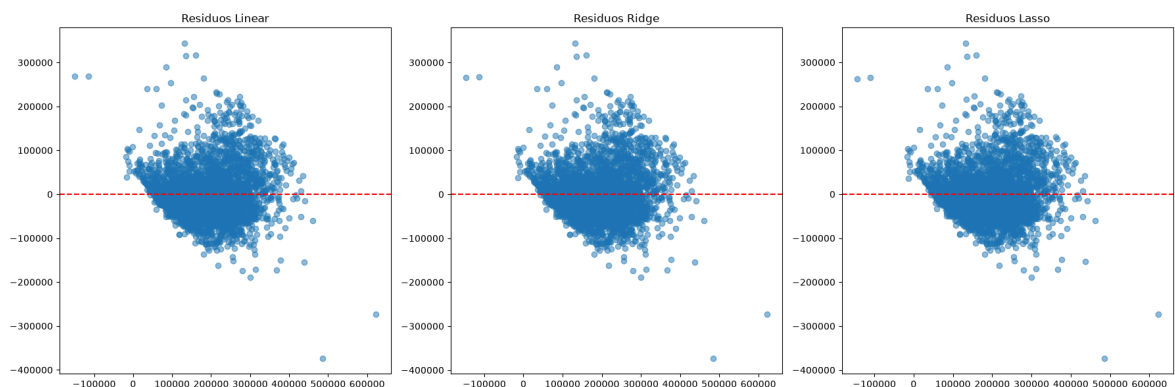
# Lasso
res_lasso = y_test - y_test_pred_lasso

ax[2].scatter(
    y_test_pred_lasso,
    res_lasso,
    alpha=0.5
)

ax[2].axhline(0, color='red', linestyle='--')
ax[2].set_title("Residuos Lasso")

plt.tight_layout()
plt.show()

```



In [48]:

```

#Curva de aprendizaje regrsión standar
from sklearn.model_selection import learning_curve
import numpy as np

train_sizes, train_scores, test_scores = learning_curve(
    linear_model,
    X_train_scaled,
    y_train,
    cv=5,
    scoring='neg_root_mean_squared_error'
)

plt.figure(figsize=(8,6))

plt.plot(

```

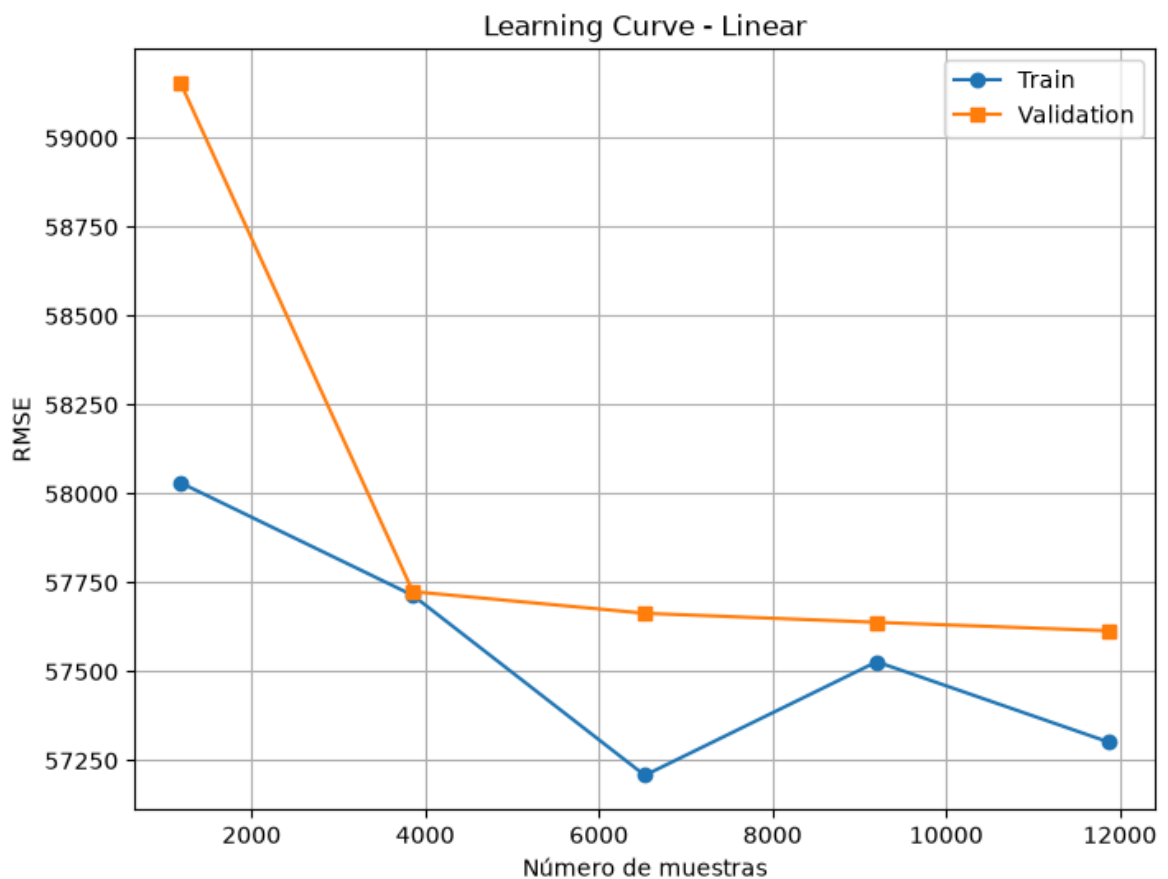
```

train_sizes,
-train_scores.mean(axis=1),
marker='o',
label='Train'
)

plt.plot(
train_sizes,
-test_scores.mean(axis=1),
marker='s',
label='Validation'
)

plt.title("Learning Curve - Linear")
plt.xlabel("Número de muestras")
plt.ylabel("RMSE")
plt.legend()
plt.grid(True)
plt.show()

```



In [49]: *#Curva de aprendizaje regresión ridge*

```

train_sizes, train_scores, test_scores = learning_curve(
    ridge_model,
    X_train_scaled,
    y_train,
    cv=5,
    scoring='neg_root_mean_squared_error'
)

plt.figure(figsize=(8,6))

plt.plot(

```

```

train_sizes,
-train_scores.mean(axis=1),
marker='o',
label='Train'
)

```

```

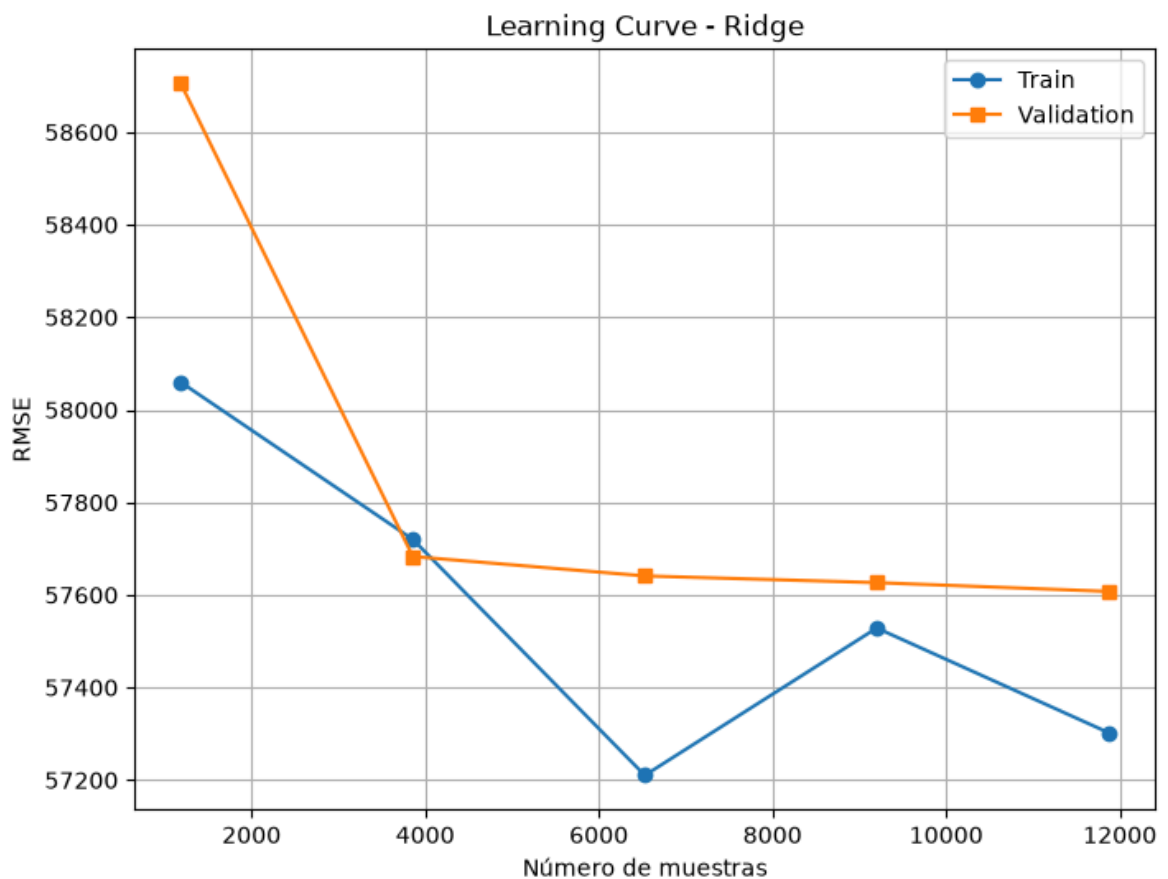
plt.plot(
train_sizes,
-test_scores.mean(axis=1),
marker='s',
label='Validation'
)

```

```

plt.title("Learning Curve - Ridge")
plt.xlabel("Número de muestras")
plt.ylabel("RMSE")
plt.legend()
plt.grid(True)
plt.show()

```



```

In [50]: #Curva de aprendizaje regresión lasso
train_sizes, train_scores, test_scores = learning_curve(
    lasso_model,
    X_train_scaled,
    y_train,
    cv=5,
    scoring='neg_root_mean_squared_error'
)

plt.figure(figsize=(8,6))

plt.plot(
    train_sizes,

```

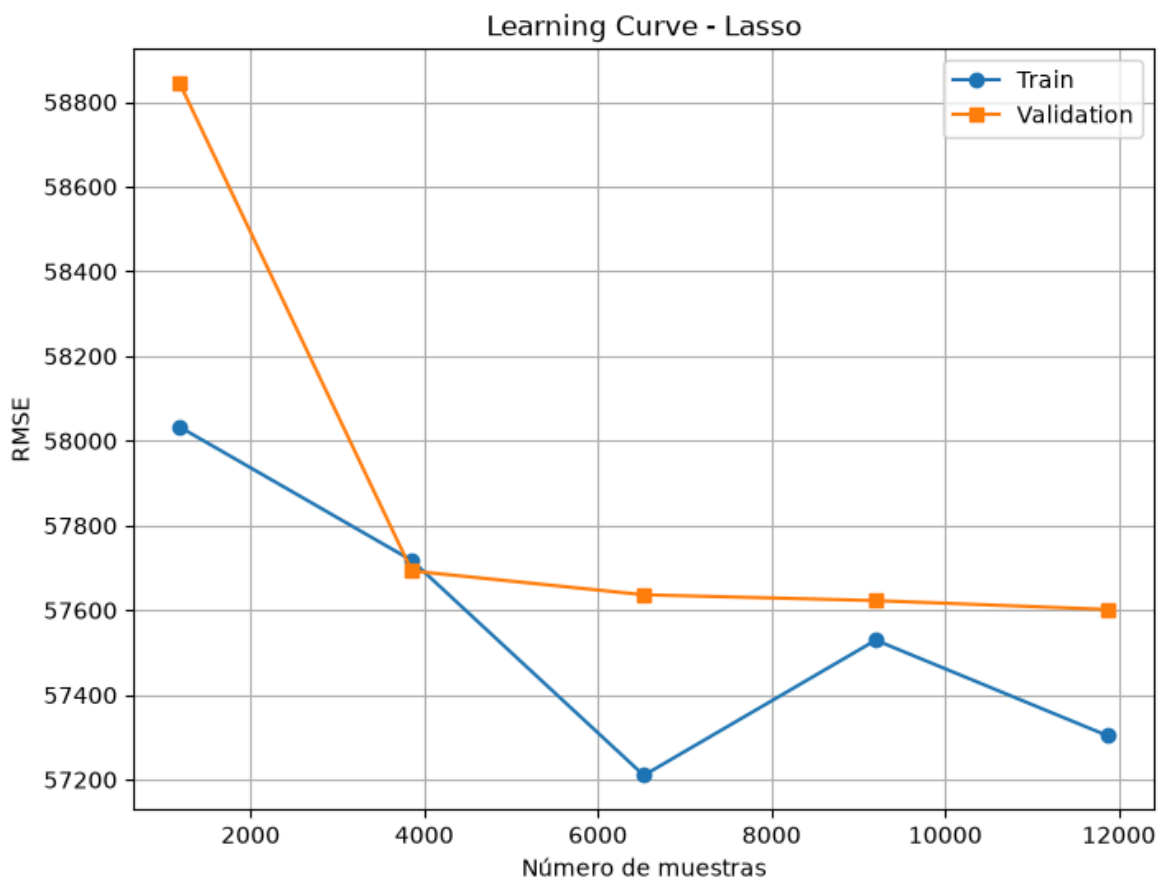
```

- train_scores.mean(axis=1),
  marker='o',
  label='Train'
)

plt.plot(
  train_sizes,
  - test_scores.mean(axis=1),
  marker='s',
  label='Validation'
)

plt.title("Learning Curve - Lasso")
plt.xlabel("Número de muestras")
plt.ylabel("RMSE")
plt.legend()
plt.grid(True)
plt.show()

```



Conclusión

- Diagnóstico de Sobreajuste (Overfitting): El Error Cuadrático Medio (RMSE) en los conjuntos de entrenamiento y prueba es sumamente similar para ambos modelos. Esto indica que la regularización aplicada es efectiva y que los modelos generalizan correctamente sin memorizar el ruido de entrenamiento.
- Comportamiento del Modelo Lasso (L1): A pesar del uso de regularización L1 (con $\alpha=50$), el modelo Lasso no ha reducido a cero ningún coeficiente. Esto sugiere que, a este nivel de penalización, todas las características de entrada conservan relevancia predictiva y no hay redundancia crítica.
- Comparativa de Modelos: El modelo Ridge (L2) presenta un rendimiento marginalmente superior en el conjunto de prueba, lo que indica que una

penalización suave distribuida sobre todas las variables (L2) es más adecuada para este conjunto de datos que la selección estricta de variables (L1).

In [51]: *### Comparación de los modelos*

```
In [52]: comparacion = pd.DataFrame({
    'Modelo': ['Linear', 'Ridge', 'Lasso'],
    'RMSE Train': [
        rmse_train_linear,
        rmse_train_ridge,
        rmse_train_lasso
    ],
    'RMSE Test': [
        rmse_test_linear,
        rmse_test_ridge,
        rmse_test_lasso
    ],
    'MAE Train': [
        mae_train_linear,
        mae_train_ridge,
        mae_train_lasso
    ],
    'MAE Test': [
        mae_test_linear,
        mae_test_ridge,
        mae_test_lasso
    ]
})

comparacion
```

```
Out[52]:
```

	Modelo	RMSE Train	RMSE Test	MAE Train	MAE Test
0	Linear	57330.006736	57261.373128	41955.672722	42267.328597
1	Ridge	57330.293705	57255.128087	41958.589108	42264.599484
2	Lasso	57331.799208	57253.640824	41953.469481	42260.119648

```
In [53]: #Deber para terminar el pippeline guardar y cargar el modelo lasso e ingresar nuevos valores por teclado y p
import joblib
from sklearn.pipeline import Pipeline

# 1. Crear el Pipeline combinando el escalador que ya entrenaste y el modelo Lasso
# Nota: Usamos lasso_model y scaler que ya definiste y ajustaste en las celdas anteriores
pipeline_lasso = Pipeline([
    ('scaler', scaler),
    ('lasso', lasso_model)
])

# 2. Guardar el pipeline en un archivo .pkl
nombre_archivo = 'pipeline_lasso_viviendas.pkl'
joblib.dump(pipeline_lasso, nombre_archivo)
print(f' Pipeline guardado exitosamente como: {nombre_archivo}')
```

3. Cargar el pipeline desde el archivo (simulando un nuevo entorno o sesión)

```
modelo_cargado = joblib.load(nombre_archivo)
print(" Pipeline cargado en memoria y listo para predecir.")
```

Pipeline guardado exitosamente como: pipeline_lasso_viviendas.pkl

Pipeline cargado en memoria y listo para predecir.

In [54]:

4. Función para ingresar nuevos valores por teclado y predecir

```
def predecir_nueva_vivienda(modelo):
    print("\n--- INGRESO DE DATOS PARA PREDECIR PRECIO DE VIVIENDA ---")
    try:
        # Solicitamos las variables base al usuario
        print("Ingresa los siguientes valores numéricos:")
        lon = float(input("1. Longitud (ej. -118.25): "))
        lat = float(input("2. Latitud (ej. 34.05): "))
        age = float(input("3. Edad mediana de la vivienda (ej. 25.0): "))
        rooms = float(input("4. Total de habitaciones (ej. 2500.0): "))
        bedrooms = float(input("5. Total de dormitorios (ej. 500.0): "))
        pop = float(input("6. Población (ej. 1200.0): "))
        households = float(input("7. Hogares/Familias (ej. 400.0): "))
        income = float(input("8. Ingreso medio (escala de 1 a 15, ej. 4.5): "))

        # Selección de categoría marítima para las variables dummy
        print("\nSelecciona la proximidad al océano:")
        print("1. NEAR BAY | 2. NEAR OCEAN | 3. ISLAND | 4. INLAND | 5. <1H OCEAN (Por defecto)")
        opcion_oceano = input("Opción (1-5): ").strip()
```

5. Cálculo automático de características diseñadas (Ingeniería de Características)

```
LA = (34.0522, -118.2437)
SF = (37.7749, -122.4194)
dist_LA = np.sqrt((lat - LA[0])**2 + (lon - LA[1])**2)
dist_SF = np.sqrt((lat - SF[0])**2 + (lon - SF[1])**2)

bedrooms_per_room = bedrooms / rooms
pop_per_household = pop / households
```

Variables Dummy (One-Hot Encoding: por defecto todas en 0 para '<1H OCEAN')

```
near_bay = 1 if opcion_oceano == '1' else 0
near_ocean = 1 if opcion_oceano == '2' else 0
island = 1 if opcion_oceano == '3' else 0
inland = 1 if opcion_oceano == '4' else 0
```

6. Crear un DataFrame con EXACTAMENTE las mismas columnas y orden que X_train

```
nuevos_datos = pd.DataFrame([
    'longitude': lon,
    'latitude': lat,
    'housing_median_age': age,
    'total_rooms': rooms,
    'total_bedrooms': bedrooms,
    'population': pop,
    'households': households,
    'median_income': income,
    'dist_to_LA': dist_LA,
    'dist_to_SF': dist_SF,
    'bedrooms_per_room': bedrooms_per_room,
    'pop_per_household': pop_per_household,
    'ocean_proximity_INLAND': inland,
    'ocean_proximity_ISLAND': island,
    'ocean_proximity_NEAR BAY': near_bay,
    'ocean_proximity_NEAR OCEAN': near_ocean
```

```

    }], columns=X.columns) # Garantiza el mismo orden que el entrenamiento

# 7. Realizar la predicción con el Pipeline cargado
precio_predicho = modelo.predict(nuevos_datos)[0]

print("\n" + "="*50)
print(f" EL PRECIO ESTIMADO DE LA VIVIENDA ES: ${precio_predicho:,.2f} USD")
print("="*50)

except ValueError:
    print("\n Error: Por favor, ingresa únicamente valores numéricos válidos (usa punto para decimales)
except Exception as e:
    print(f"\n Ocurrió un error inesperado: {e}")

# Llamar a la función utilizando el modelo cargado desde el archivo .pkl
predecir_nueva_vivienda(modelo_cargado)

```

--- 🏠 INGRESO DE DATOS PARA PREDECIR PRECIO DE VIVIENDA ---

Ingresa los siguientes valores numéricos:

Selecciona la proximidad al océano:

1. NEAR BAY | 2. NEAR OCEAN | 3. ISLAND | 4. INLAND | 5. <1H OCEAN (Por defecto)

=====

💰 EL PRECIO ESTIMADO DE LA VIVIENDA ES: \$464,248.59 USD

=====

In []: